

KPI Assurance and LLMs for Intent-Based Management

Kristina Dzeparoska

Department of Electrical and Computer Engineering
University of Toronto
Toronto, Ontario
kristina.dzeparoska@mail.utoronto.ca

Alberto Leon-Garcia

Department of Electrical and Computer Engineering
University of Toronto
Toronto, Ontario
alberto.leongarcia@utoronto.ca

Abstract—Intent-Based Management provides a shift in network management by automating the alignment of network operations with business objectives. However, primary challenges include: 1) intent processing (translate, decompose and identify the logic to fulfill the intent), and 2) ensuring intent conformance (ongoing adaptation of the logic to ensure the intent is met, considering dynamic conditions). We use a 3-tier Large Language Model (LLM) pipeline to convert intents into Policy Trees, that are then executed using closed control loop automation. In this paper, we focus on assurance that is tasked with continuous monitoring, verification, and validation of the operational state, and corrective actions to ensure conformance with the target objectives. To do so, we use a generic LLM (OpenAI's GPT) with in-context learning and well-established decision-making algorithms (feedback controllers) to determine assurance actions and remediate intent deviation. We show that AI-driven policies can support intent fulfillment and assurance, and we discuss current limitations, benefits, and future directions to address critical challenges in using AI for network management, towards improved generalizability, scalability, and overall trustworthiness.

Index Terms—intent assurance, fulfillment, policy abstraction, few-shot learning, intent decomposition, LLM

I. INTRODUCTION

The rapid growth and increasing demands of services and applications led to increasingly complex infrastructures that are difficult to manage at scale. Modern communication networks rely heavily on software for the design, operation, and management of its resources. The complexity of these systems, consisting of heterogeneous and distributed components, necessitates advanced management systems capable of automation and dynamic adaptation to changing network conditions and requirements. Autonomic systems simplify network operations and reduce the need for manual intervention; however, they cannot automatically understand the provider's goals, so the concept of intent helps guide their operations.

Intent-based management aims to automate network operations by realizing high-level intents into configurations, while ensuring that the operational state matches closely the target state. These core functions are known as fulfillment and assurance [1]. To formalize intents, and derive the logic to fulfill and assure, modeling is essential. Policy-based languages can capture requirements at various levels of abstraction, and provide the means for explanatory management, as the actions are policy-driven. Intent and policy share a foundation in that both

represent a higher level of network abstraction—dependent of device-specific details, but they differ fundamentally. Policies define rule-based actions (event-condition-action structures) without specifying desired outcomes, while intents express high-level, outcome-focused goals, leaving the methods to be automatically “**learned or derived from the intent**”.

We take a hierarchical policy approach and use *declarative, definitive, and imperative policies* that apply at different levels of abstraction through the hierarchy to decompose intents into MAPE-K policies (*Monitor, Analyze, Plan, Execute, Knowledge*) [2]. The result is a holistic **Policy Tree** (ordered set of policies) that defines the logic to fulfill and assure intents and mirrors human decision-making steps to realize intents, all the way to configuration changes. *This potentially allows our framework to be applied to other environments that can be abstracted into the form of a network of resources, e.g., smart cities, healthcare, public safety, and transportation networks.*

We dubbed our system “Emergence” due to its ability to enable network configuration and operation to emerge from intents. To intelligently and automatically decompose intents into comprehensive policy trees using our hierarchical approach, we leverage a Large Language Model (LLM) pipeline with in-context learning. The pipeline classifies intents, generates policies, and validates the policy logic to ensure alignment with the intent [3]. We map policies to domain-specific APIs and automate policy execution through hierarchical closed control loops and Finite State Machines [4]. Supervisory loops receive intents and policy trees, and then distribute each policy tree to a subordinate loop that specializes in the given service—allowing to parallelize executions without dependencies. To close the loop and maintain the intent during its life-cycle, monitoring intent drift is essential to capture deviations, guide corrective actions, and support learning over time for predictive behavior. To do so, we monitor and assess KPIs and metrics, and use AI-driven assurance policies via few-shot learning [5]. For simple issues, we can rely on the LLM to generate assurance policies (e.g., a policy to restart a resource), however, for complex tasks like resource scaling, the model faces challenges in finding an optimal solution.

To address this limitation, we propose a control model, and showcase the practical application of control systems for Intent Assurance. By providing drift-related context as input,

and access to external services, such as a controller, the LLM reasons about remediation actions, and generates the required policies independently or using the external solution. This approach can be generalized and extended through Retrieval-Augmented Generation (RAG), allowing the LLM to access diverse algorithms and services for complex computations and seamlessly generate assurance policies.

We evaluate container autoscaling, dynamic allocation of CPU and RAM, in response to varying workloads. We fulfill and assure a use-case intent with an application SLA using AI-driven policies, supported by a control model. Our approach is novel because it involves hierarchical and comprehensive Policy Trees, that provide the necessary steps to realize an intent, rather than directly converting the intent into a configuration policy (e.g., P4, BGP, 5G [6]–[14]). Our contributions are:

- Evaluate a controller-based design for assurance.
- Leverage generative models (LLMs) to fulfill and assure intents when intent drift occurs.
- Discuss limitations and opportunities to enhance scalability, adaptability, and trustworthiness of AI applications in intent-based management systems.

In Section III we describe our assurance framework, including KPI assessment, and the LLM pipeline. Then, in Section IV, we derive a controller mechanism from a discrete-time feedback controller to support autoscaling. This approach helps to resolve LLM limitations by embedding or allowing access to different algorithms or tools to assist when computing optimal solutions. A detailed evaluation of the controller, as well as the fulfillment and assurance of an intent is given in Section V. Finally, in Sections VI and VII we provide a discussion on limitations and opportunities for trustworthy applications of AI in management, and our future work.

II. BACKGROUND AND RELATED WORK

Intent drift refers to the gradual deviation between the operational and target state, due to changing conditions. Detecting and resolving the drift is essential for ensuring compliance with the intended behavior [1], e.g., [15] monitors and assures network slices by triggering corrective measures. RFC 9417 [16] presents the SAIN (Service Assurance for Intent-Based Networking) architecture, which uses assurance graphs that can be decomposed into independently managed components. The graph structure allows the service instance to be monitored along with its dependencies, however computing health scores and measuring drift is beyond its current scope [17].

The comprehensive survey [18] outlines intent challenges and key components. Assurance responses are categorized into reactive and proactive, with actions such as flow and VM migration, scaling, alerting, and failure recovery. Open challenges include intent expression, use of AI, lack of datasets, flexible and customized monitoring, standardization, multi-domain, and security [19]. A key challenge lies in correlating information technology KPIs with operational technology KPIs, such as those in Industry 4.0, emphasizing the need for standardized KPIs to ensure consistency and interoperability. Other challenges are continuous learning and adaptation in AI

models, balancing automation with human oversight for critical decision-making, and managing conflict and redundancy.

Numerous approaches have been investigated, including heuristics [20], optimization techniques [21], rule-based systems [22], and Machine Learning and Artificial Intelligence [23]–[26]. These range from natural language processing (NLP) to inference (e.g., time series models [27] to predict CPU surges, and determine actions such as VM migration or scaling) and predictive analytics, forecasting deviations and recommending assurance actions. For example, detecting when flow migrations should be done, using Random Forest [28], finding optimal routes using Reinforcement Learning or neural nets [29]–[31], resource scaling for VNFs in service function chains based on CPU utilization using Multilayer Perceptron classifier [32] or LSTM-based [33]. Still, predefined workflows can facilitate some tasks as well, for example NETASSISTANT [34] is an AI enabled diagnosis framework, taking as input questions in natural language, to provide network diagnosis workflows. Our work similarly tries to create AI workflows (via Policy Trees) to realize the intent.

III. INTENT ASSURANCE AND INTENT DRIFT

Figure 1 shows the main steps in Fulfillment and Assurance, with a focus on the latter. We assure intents by formalizing the KPIs (Section III-A1) to obtain the Target KPIs. Then, we monitor and assess the corresponding operational KPIs by applying different policy functions to detect intent drift (Sections III-A2, III-A3, III-A4).

The KPI and drift analysis (Section III-A) highlights the need to constantly evaluate and adjust operational KPIs to ensure they remain aligned with target KPIs. The intent drift vector can provide insight into how far and in which direction the system is deviating from its desired state, but this is only the first step in addressing the misalignment. Given that the LLM sometimes struggles with quantitative actions, we need an efficient method to compute corrective actions, leading us to a controller-based assurance feedback approach (Section IV)—that allows us to adjust resources in response to real-time variations in KPIs. The LLM can access the controller when drift is detected, and use the computed solution to generate a resource scaling policy.

A. KPI Analysis Methodology for Intent Assurance

Defining KPIs is a complex topic, yet KPIs have long been a cornerstone of network management, with definitions available through IETF standards, industry guidelines, etc. A standardized KPI dataset is highly beneficial to ensure consistency and interoperability. In our prior work, we defined a set of KPIs and proposed a generalized KPI framework that maps these to intents based on type, facilitating alignment between intent objectives and measurable outcomes [5], [35].

1) *Formalize Intent with KPIs*: We use few-shot training to guide the LLM in extracting the entities and KPIs that define the intent's objectives. This helps identify the parameters that need to be met for the intent to be deemed healthy. The input includes intent, type, policy tree. For the use-case intent:

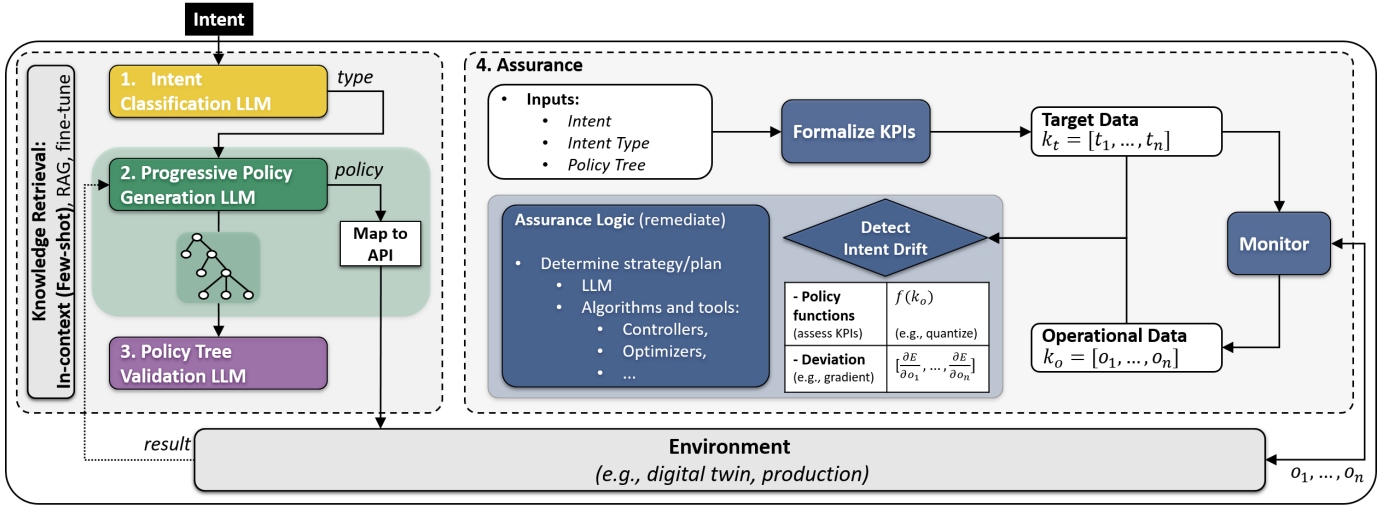


Fig. 1: Overview of the main components for decomposing and assuring intents. The intent is first classified, then the policy tree is generated and validated. Once the intent is deployed, the main steps for Assurance include: formalize KPIs, monitor the operational KPIs and assess against targets, detect intent drift, and determine a strategy to assure the intent. Last, invoke the policy generation module once a solution is found, to enact the necessary change(s) onto the environment.

“Create a containerized web-server application using image *appl* in region *us-central1*, in a cluster called *my-cluster*, such that the application has an upper-bound response time of 0.6s. Schedule a default health check for the application and send the results to *app_manager*.”, the extracted KPI vector is: $\{ \text{“response_time”}: “\leq 0.6s”, \text{“health”}: I \}$.

2) *KPI Quantization*: To determine if an operational KPI deviates from its target value, we apply a quantization function to classify the KPI. This allows us to quickly assess KPIs.

3) *Policy functions for logical KPI assessment*: To evaluate KPIs, we assess and combine multiple metrics and/or KPIs, by applying functions that are specified by policies. This enables generic management, as the functions are defined using a policy-based approach. For example, we model the health of a resource (e.g., VM, container) as a vector of CPU, RAM, and storage utilization, network and resource status, and apply a policy function (f), e.g., quantize and apply Kleene’s logic. By providing the necessary context and examples, LLMs are able to generate these KPI-related policies, as well as review the policy execution results for drift analysis and then generate assurance actions either independently, or using external services for the computation.

4) *Intent drift*: The intent drift represents the deviation between the operational and target KPIs, and identifies the need for corrective actions. It can be expressed in various ways, such as a ratio, or the relative values of the actual measured performance KPI and its target value, or the difference. The drift analysis helps us identify deviations and offers critical insights into both the direction and magnitude of necessary adjustments, however it does not directly translate into corrective actions. For this, we leverage an LLM to reason about the actions, supported by a controller-based approach for assurance, as described next.

B. Generating policies using LLMs and in-context learning

We proposed a novel framework for translating intents into actionable policies using LLMs, leveraging the MAPE-K closed control loop as a guiding structure [3]. The 3-tier LLM pipeline uses an LLM (OpenAI’s GPT) with in-context learning to progressively decompose intents into policies using state feedback. First, we classify intents into predefined types, then, generate the policies progressively: after a policy is generated, it is mapped to an API for execution, and the result is fed back to the LLM (Figure 1). For example, when executing a policy that checks for an available resource, the feedback includes availability information. This enhances the LLM’s ability to adapt to changing conditions, and allows one to embed algorithms or decision-making rules into the process.

To have the LLM decompose intents, and understand the environment in which the generated policies need to operate, the input includes: behavior instruction; descriptions and examples: *policy model*, *actions* (e.g., get, avail, reserve, create, configure), *resources* (e.g., vm, container, link, path), and *constraints* (e.g., zone, count, frequency, size); *action sequence* for common intent categories (resource creation, service deployment, topology discovery); and, a few *intent-to-policy examples*. The policies are represented as JSON objects to enable seamless integration with the APIs. The output from the progressive decomposition is a Policy Tree that we validate using the LLM by incorporating examples of policy sequences and their corrections into the prompt, such as omissions, attribute errors, or sequencing issues. For assurance, we provide contextual information, including the intent, policy tree, and current conditions as input to the LLM to reason and generate assurance policies. To tackle complex problems, the LLM uses external services such as controllers, optimizers to obtain the solution and create the policy.

IV. CONTROLLER-BASED ASSURANCE FEEDBACK

To ensure that the system continuously aligns with its target performance, we require an active mechanism that not only identifies drift, but also resolves it in real-time. Relying solely on an LLM may not always yield the optimal action, especially when complex computations are required. Instead, well-established algorithms such as a controller-based approach can support the LLM during assurance. For this, we use the method presented in [36], from which we derive a controller-based equation for resource allocation, which allows us to operate without the actual dynamic control system. Together, the drift analysis informs us about the evolving behavior, while the controller helps ensure that the system remains compliant with the intent by computing the necessary changes, which are then translated into policies by the LLM for execution.

A controller can maintain a desired performance metric (e.g., response time), under dynamic request rate, by computing the resources that need to be adjusted, (e.g., CPU, memory). The evolution over the discrete time index k of the response time $\tau_r(k)$, as an effect of the assigned cores $c(k)$ and the request rate $r(k)$, is ruled by the nonlinear, time-invariant dynamic system that corresponds to the block diagram.

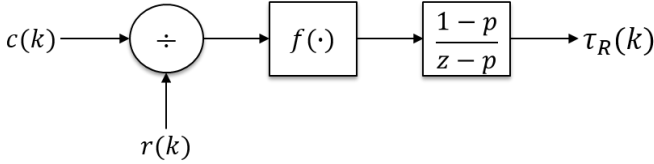


Fig. 2: Dynamic control system, as in [36].

$$\begin{cases} \tilde{u}(k) = f\left(\frac{c(k)}{r(k)}\right) \\ \tau_r(k) = p\tau_r(k-1) + (1-p)\tilde{u}(k-1) \end{cases} \quad (2)$$

where $p \in [0, 1)$ is the single pole of the system estimated with step response analysis.

We leverage the function in [36] that defines the steady state delay relationship between the CPU cores and request rate as:

$$f\left(\frac{c(k)}{r(k)}\right) = c_1 + \frac{c_2}{1 + c_3 \frac{c(k)}{r(k)}} \quad (3)$$

where parameters c_1 , c_2 , and c_3 are obtained through profiling. The function represents either the maximum or minimum delay, considering a finite buffer. That is, as $r_t \rightarrow \infty$, the term $c_3 \frac{c_t}{r_t} \rightarrow 0$, and the asymptote is $c_1 + c_2$ which corresponds to the maximum delay. Similarly, if $r_t \rightarrow 0$, the term $c_3 \frac{c_t}{r_t} \rightarrow \infty$, then the function provides the minimum delay, given by c_1 .

A. Deriving a controller-based scaling approach

We solve the equation for the number of cores. If we consider n steps, then the function provides the average delay defined as: $f_1\left(\frac{c}{r}\right) = \text{avg}D\left(\frac{c}{r}\right)$.

Let $\bar{D}_t$ be the actual delay, where c_t is fixed, and let D_{TR} be the target delay, defined as:

$$D_{TR} = f\left(\frac{c_{\text{new}}}{r_t}\right) \quad (4)$$

Given that $\bar{D}_t$ is measured, and c_t is fixed during the measurements, allows us to solve for r_t . Then, by substituting for r_t , we can solve for c_{new} . We start with Equation 3, and solve for r_t , where as $r_t \rightarrow \infty$, the term $c_3 \frac{c_t}{r_t} \rightarrow 0$, and if c_1 is negligible, then $\bar{D}_t = c_2$, i.e., c_2 is the maximum delay, considering a finite buffer.

$$r_t = \frac{c_3 c_t}{\frac{c_2}{\bar{D}_t - c_1} - 1} \quad (5)$$

Next, we solve for c_{new} :

$$c_{\text{new}} = \frac{r_t}{c_3} \left(\frac{c_2}{D_{TR} - c_1} - 1 \right) \quad (6)$$

And, we substitute for r_t to obtain c_{new} in terms of the delays (c_t is technically c_{old}):

$$c_{\text{new}} = c_t \left(\frac{D_t - c_1}{D_{TR} - c_1} \right) \left(\frac{c_2 - (D_{TR} - c_1)}{c_2 - (D_t - c_1)} \right) \quad (7)$$

As can be seen, there are two kinds of drifts that appear in Equation 7. The drift does not appear directly, since we have the terms in the form of: $D - c$. Suppose we redefine things, so we can treat c_1 as 0. Then, the delays subtracted from the constant c_1 can be thought of as a kind of drift, excluding the constant term. So, we define:

$$D_t \rightarrow D_t^* = D_t - c_1 \quad (8)$$

$$D_{TR} \rightarrow D_{TR}^* = D_{TR} - c_1 \quad (9)$$

And, once we substitute for the above expressions, we obtain c_{new} as:

$$c_{\text{new}} = c_t \left(\frac{D_t^*}{D_{TR}^*} \right) \left(\frac{c_2 - D_{TR}^*}{c_2 - D_t^* - p \times \sigma} \right) \quad (10)$$

When the variation is high between the average delays, we have less certainty of the mean estimate, and that is why we take the standard deviation (σ) into account, and p is a scaling parameter that needs to be profiled (in our case we set $p = 0.5$). The deviation here is a ratio, and, we essentially have two terms, the first term is relative to the ratio of the delays, and the second one relative to how close we are to the ceiling (c_2). Suppose $p = 0$, then the expression $(c_2 - D_{TR}^*)$ becomes a margin, i.e., how far we are from the ceiling, and $(c_2 - D_t^*)$ is the ceiling with respect to the current delay.

By rearranging the terms, we can express c_{new} as:

$$c_{\text{new}} = c_t \left(\frac{\frac{c_2}{D_{TR}^*} - 1}{\frac{c_2 - p \times \sigma}{D_t^*} - 1} \right) \quad (11)$$

where we see that c_{new} is equal to the previous core allocation (c_t) that was fixed during time t , and we have ratios based on the measured and target delay. Similarly, if $p = 0$, the expression $(\frac{c_2}{D_{TR}^*} - 1)$ is kind of the margin for the target.

B. Why monitor Intent Drift even with a Controller in place?

The controller aims to correct the immediate KPI deviation, bringing D_t^* in line with D_{TR}^* . However, drift monitoring provides valuable insights into how fast and in what direction the KPI is changing over time. By tracking the drift, we can assess the controller's performance and detect whether tuning is needed to prevent or correct systemic issues. In addition:

- 1) *Unmanaged KPIs*: Controllers manage specific KPIs, whereas drift monitoring captures the broader environment.
- 2) *Controllers Have Limits*: Controllers typically operate within predefined limits and may have a finite number of CPU cores or other resources to allocate. In situations where the load increases significantly beyond the controller's capacity, drift can still occur. By monitoring drift, we can detect when the controller is no longer able to manage and alert higher-level processes to take action, such as executing admission control, shedding traffic, obtaining additional resources, etc.
- 3) *Unexpected Failures and Anomalies*: If a controller malfunctions or a system anomaly occurs (software bug, unexpected workload pattern), drift may reappear and alert the system for corrective actions before significant impacts occur.
- 4) *Learning and Adaptation*: Historical KPI and drift data can provide valuable insights for learning and adapting the assurance mechanism over time. By analyzing patterns, we can identify relationships between different KPIs and assurance actions. This analysis can inform more effective and predictive responses, allowing the system to adjust proactively based on expected trends, rather than reacting after violations occur. In addition, the controller parameters may need adjusting, and considering a digital twin, we can perform offline, in-parallel tuning of the parameters using the historical data.

V. EVALUATION AND RESULTS

Next, we briefly discuss the LLM-based policy generation for intent fulfillment, followed by intent assurance, including intent drift detection under dynamic workloads. Finally, we provide a comparison between the derived and original controller approach, as described in the previous section.

A. Environment setup

We conduct our evaluation using Google Kubernetes Engine on Google Cloud Platform (GCP). This setup enables dynamic vertical scaling of the CPU and memory for the container. We provision 512MB RAM per CPU. We analyze the bottleneck produced by the processor, and measure the response time (queuing and processing) as the elapsed time from when the request arrives, until a response is sent back. To improve CPU utilization, the web application uses Gunicorn [37]. For monitoring, we set Prometheus with a 5s scrape interval.

B. LLM-based Progressive Policy Tree Generation

To deploy the intent, we pass it through the LLM pipeline to be classified into *create resource*, *deploy service*, *health check*; then, decomposed into a set of policies, and validated. The progressive policy generation LLM is trained to output policies in JSON format for easy integration into code, for example,

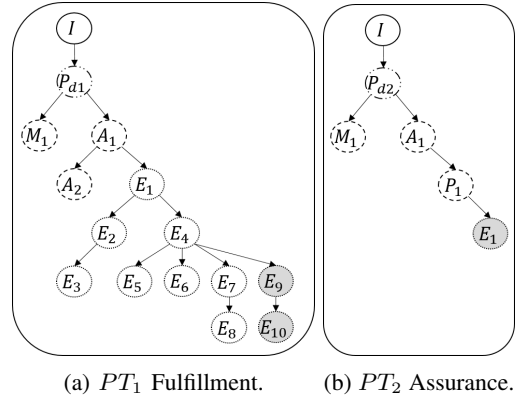


Fig. 3: Policy Trees for the intent use-case.

{“action”:“get”, “resource”:“compute”, “type”:“machine-types”, “zone”:“us-central1-a”} corresponds to M_1 in Table I (determines the VM types in GCP). Figure 3 shows the generated policy trees, PT_1 for intent deployment, and PT_2 for health check and assurance. PT_1 contains policies to check available machine types in GCP, image availability, creating a cluster, deploying the application container, deploying a Prometheus container for monitoring, and an assurance policy using the controller function from Equation 11.

C. Assurance through Intent Drift and Corrective Actions

We evaluate reactive and proactive assurance strategies, both supported by the health check. We provide the reactive evaluation first in V-C1, and in Section V-E we evaluate the proactive one through a comparison between our derived Equation 11 and the dynamic control system as in [36].

The reactive strategy involves PT_1 without E_9 and E_{10} . As part of the health check, we monitor the KPIs, and feed monitoring and drift data to the LLM, to determine if corrective actions are necessary. For example, if response time drift is detected, the LLM can access the controller to compute the new allocation, and then generate a policy to scale, e.g., E_1 is created under PT_2 . The proactive strategy is PT_1 with E_9 and E_{10} to enable automatic scaling. The controller actively computes the allocation (E_9), and the LLM uses the result to generate the scaling policy (E_{10}).

The health check is not limited to scaling policies only, and as such the LLM may generate other policies to assure the intent. For example, if the container status indicates *Error*, then a policy to recreate the container will be generated and executed under PT_2 . In fact, regardless of the strategy, the health check (PT_2)¹ serves to monitor, analyze, plan and execute changes based on the health check results. To do so, we leverage LLMs to reason based on the requirements, current results and conditions, and accordingly generate policies progressively based on feedback to fulfill and assure intents.

¹The policies are omitted for brevity, but include monitoring, analysis, planning, and execution. For example, execution policies include actions such as scale, restart, recreate container, deploy, update, configure, etc.

ID	Action	Resource	Constraint
M ₁	get	[compute]	type=machine-types, zone=us-central1-a
A ₁	avail	[compute]	machine_type=n1-highcpu-8, zone=us-central1-a
A ₂	avail	[image]	name=app1
E ₁	create	[container_cluster]	name=my-cluster, num_nodes=1, machine_type=n1-highcpu-8, zone=us-central1-a
E ₂	deploy	[container]	config_file=deployment.yaml, image=app1, label=my-app, cluster=my-cluster, zone=us-central1-a
E ₃	expose	[container]	deployment=app1-deployment, type=LoadBalancer, port=80, target_port=80, cluster=my-cluster, zone=us-central1-a
E ₄	deploy	[container]	config_file=prometheus_deployment.yaml, regex=my-app, cluster=my-cluster, zone=us-central1-a
E ₅	get	[my-cluster]	metric=IP, zone=us-central1-a, zone=us-central1-a
E ₆	get	[prometheus-deployment]	metric=port, cluster=my-cluster, zone=us-central1-a
E ₇	healthcheck	[app1-deployment]	cluster=my-cluster, zone=us-central1-a, output=app_manager
E ₈	schedule	[E ₇]	frequency=5min
E ₉	assure	[app1-deployment]	response_time ≤ 0.6s, method=controller, cluster=my-cluster, zone=us-central1-a, output=app_manager
E ₁₀	scale	[app1-deployment]	CPU=E ₉ , RAM=E ₉ , cluster=my-cluster, zone=us-central1-a
E ₁	scale	[app1-deployment]	CPU=2, RAM=1024, cluster=my-cluster, zone=us-central1-a

TABLE I: Policies for the fulfillment and assurance of the Intent use-case, using our Policy Model defined in [2] where a policy is modeled as: $\vec{P} = (D, E, A, \vec{C})$ and $\vec{C} = (\vec{R}, \vec{T}, \vec{S})$, i.e., definer, entity that enforces the policy, action, vector of constraints that includes: resource, temporal and spatial constraints. For brevity, we omit D and E from the policies.

1) *Drift Detection and Reactive Assurance*: We evaluate the experiment using a high-load that lasts for 15 minutes, and during this time, we generate three varying loads, starting with 200, 800 and 400 r/s, each lasting for 5 minutes. As part of the health check, we monitor the application's metrics and KPIs, e.g., response time, CPU, RAM, storage utilization, container status. In Figure 4 we show the response time, core allocation, and derivatives of the Mean Squared Error and Mean Absolute Error with respect to the response time. When the second load starts (800 r/s, around 18:15min), the response time goes above the threshold, i.e., drift occurs, indicating adjustments are necessary. During this time, we have the following target and operational values for the response time: $k_l^t = \{RT : 0.6\}$; $k_l^o = \{RT : 0.63\}$; and, using MSE, the drift is: $\nabla E = \{\nabla RT : 0.1\}$. If we simply need to know if we are above or below the threshold, we can use MAE, in which case: $\nabla E = \{\nabla RT : 1\}$. The LLM analyzes the results and as a result of the response time drift, it initiates the controller to compute the new allocation, and uses the output to generate policy E_1 to scale the container to 2 CPUs, and 1024MB RAM. As can be seen in Figure 4, with the reactive approach, the SLA is violated for a brief period, that is the time to detect, invoke the controller, and generate and enforce the policy to scale.

D. Controller and Workload setup

For the controller, we take the 30 second average of the respective metrics, e.g., response and request rate. The controller uses this data to determine the number of CPUs and memory to be allocated to the web server container. We measured the average time it takes to terminate and scale the container, as well as to reload the Gunikorn configuration to update the number of workers, and found it to be less than 1 minute. We configure the control interval to be 50 steps (≈ 1 minute), and we use the most recent 30-second average in order to compute the cores during each control interval. Per each allocated core, we allocate 512MB of memory to the web server container. For the web server application, we profiled the parameters (c_1, c_2, c_3), and set the minimum and maximum number of cores to 1 and 6, respectively. We set the SLA, i.e., response

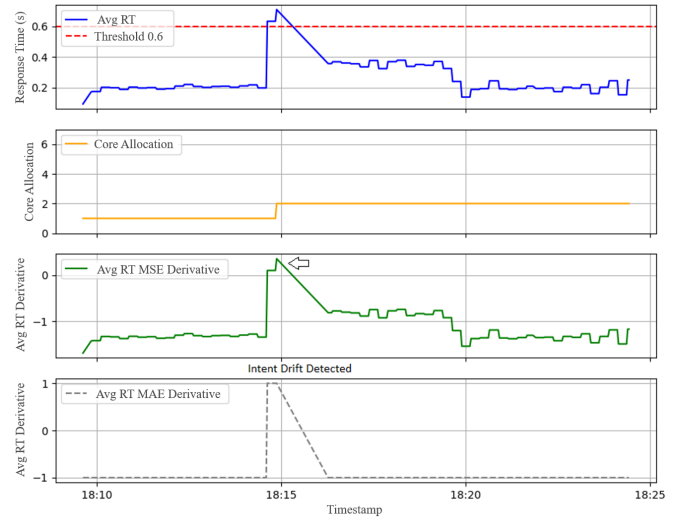


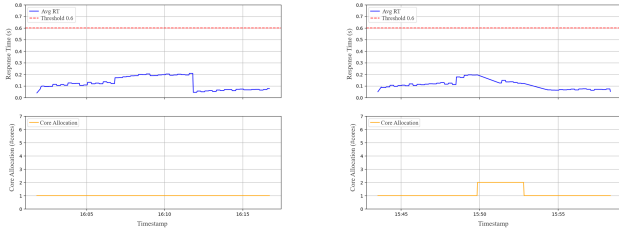
Fig. 4: Second Policy Tree Assurance approach.

time to be 0.5s, which is about 15% less than the actual SLA (0.6s). This helps ensure that the controller can maintain the response time without violations occurring.

For load testing, we use an external virtual machine with Apache JMeter [38] and evaluate the performance under four workloads (each lasts a total of 15 minutes: with 3 varying request rates, 5 minutes each): Low-load, Mid-load, High-load, and Overload, provided in Table II. At the beginning of each workload, the web server container has 1 Core, 512MB RAM.

Request Rate \ Time Interval	Time Interval		
	0-5 min	5-10 min	10-15 min
Low-load	100 r/s	200 r/s	50 r/s
Mid-load	200 r/s	400 r/s	600 r/s
High-load	200 r/s	800 r/s	400 r/s
Overload	400 r/s	1200 r/s	600 r/s

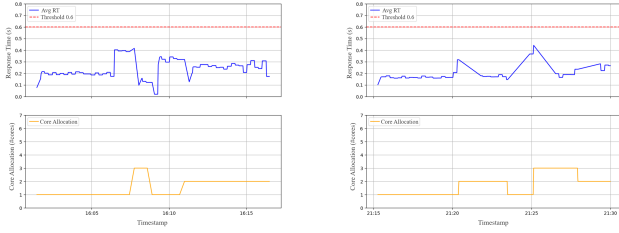
TABLE II: Evaluation workloads for each 15-minute interval.



(a) Approach 1.

(b) Approach 2.

Fig. 5: Response Time and Core allocation under low-load.



(a) Approach 1.

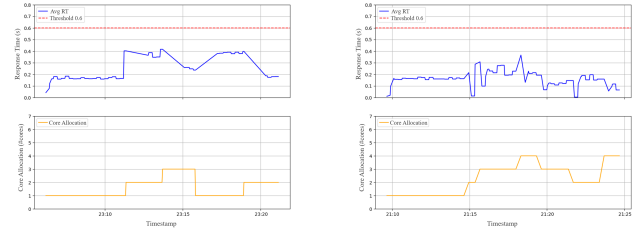
(b) Approach 2.

Fig. 6: Response Time and Core allocation - medium-load.

E. Evaluation Results: performance comparison

We evaluate the proactive approach next, focusing on the performance comparison between our direct method using the derived equation (Approach 1) and the method from [36] (Approach 2). During low-load (Figures 5a and 5b) for both approaches, the observed RT fluctuates between 0.05 and 0.2, on average ≈ 0.15 , leaving a significant margin compared to the threshold ($0.6 - 0.15 \approx 0.45$). Thus, the observed RT is around 25% of the target. The original method scales to 2 cores, whereas our derived method uses 1 core (while the RT remains well below the threshold, indicating that the minimal core allocation is sufficient under low-load conditions). During medium-load (Figures 6a and 6b), using our method, the observed RT fluctuates between 0.2 and 0.4, on average ≈ 0.25 . The margin compared to the threshold is ≈ 0.35 , and the observed RT is $\approx 40\%$ of the target. Using the original method, the RT ranges between 0.2 and 0.4, on average ≈ 0.2 . This leaves a margin of ≈ 0.4 , similar to our method, and the observed RT is $\approx 35\%$ of the target. Core allocation ranges from 1 to 3 cores for both methods.

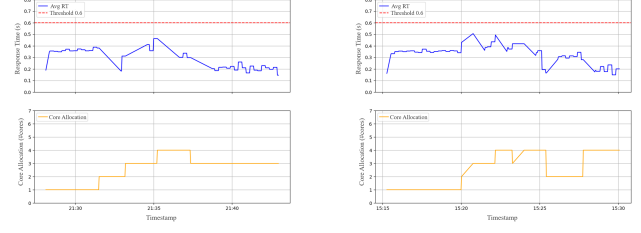
During high-load (Figures 7a and 7b), both approaches maintain the SLA, with our method allocating less resources on average during the evaluation period, with a maximum of 3 CPUs compared to 4 CPUs for the original method. For our method, the observed RT fluctuates between 0.2 and 0.5, averaging around 0.3. The margin compared to the threshold is around 0.3, and the observed RT is $\approx 50\%$ of the target. The core allocation is more dynamic (between 1 and 3 cores), mostly using 1 to 2 cores. For the original method, the observed RT is similar, fluctuating between 0.1



(a) Approach 1.

(b) Approach 2.

Fig. 7: Response Time and Core allocation under high-load.



(a) Approach 1.

(b) Approach 2.

Fig. 8: Response Time and Core allocation under over-load.

and 0.4, on average ≈ 0.2 , margin around ≈ 0.4 , and the RT is $\approx 35\%$ of the target due to higher core allocation. Last, during over-load (Figures 8a and 8b), in both approaches, the observed RT fluctuates between 0.2 and 0.5, averaging around 0.3. The margin compared to the threshold is ≈ 0.3 , meaning the observed RT is $\approx 50\%$ of the target. The core allocation using our method is slightly more conservative (less resources), however both use between 1 and 4 cores.

In summary, our method has a slightly more conservative allocation strategy (uses less resources, while still maintaining the SLA), whereas the original method allocates more gradually and is more sensitive to the request rate. Our key takeaway is that control-based systems can be adapted to handle different performance characteristics, and can be used by the LLM to determine the assurance policies, enabling intent-based systems to remain aligned with their goals.

VI. DISCUSSION: LIMITATIONS AND OPPORTUNITIES

We now discuss our insights and lessons learned [39], as well as limitations, and opportunities to enhance the application of AI in network management and intent-based systems.

A. Generalizability and Scalability

Our hierarchical policy approach allows us to decompose the intent processing steps at different levels of abstraction, applicable to different parts of the MAPE-k loop. This simplifies and reduces the action space, making it easier for the LLM to learn the effects of different actions. Moreover, we can generalize to different topologies, without topology-specific LLM training. This is because we consider the intent problem using a top-down approach, and once at the planning stage,

if topology information is needed, we supply it as context; however, most intents do not require the complete topology, rather a service chain, or path information.

Towards the ability to generalize across a diverse range of intents, we classify intents and feed few-shot prompts related to the intent type. This can be further enhanced through RAG, by allowing the LLM to retrieve additional context and to deal with new intents. As part of the context, knowledge graphs can be used to organize hierarchical data.

Scalability is supported by the model's ability to generalize from a broad range of examples and contexts. However, given the LLM's context limitation, we first classify intents and then provide the relevant information, thereby reducing the amount of data as input. In terms of policy scalability, the smaller the policy tree (15–20 policies), the more accurate the LLM's performance. For this, we decompose complex intents into multiple policy trees (per service), and then validate the intent by merging and processing the trees together, using a comprehensive instruction prompt that covers the entire cycle.

A challenge in both is the lack of standardized datasets and the difficulty in obtaining comprehensive data to train these models effectively. This makes it difficult to generalize the models across different intents and contexts, and further compounds the challenge of scaling these models.

B. Adaptability and Standardized Datasets

Adaptability is related to the quality and extent of the data that is made available to the model. For example, by providing the policies derived from an intent, knowledge about resources, performance indicators and their interrelations, the LLM can identify the KPIs to monitor and assess. That is, the model's effectiveness hinges on the depth of the knowledge base – the more data about the KPIs, metrics, and underlying dependencies, the better the model can perform for assurance. Similarly, for fulfillment, data pertaining to the available resources, actions (APIs), and sequences is required. At the same time, feeding too much information in a single prompt can be overwhelming and may lead to irrelevant KPIs or metrics. *A more effective approach to enhance the adaptability and accuracy of LLMs is to provide targeted prompts that are closely aligned with the specific intent type.* Knowledge graphs can complement this, by providing structured connections between KPIs, metrics, and resources. For instance, a graph can map out how different metrics, such as CPU, memory and bandwidth usage, contribute to higher-level performance indicators like availability.

During inference, additional data, such as current conditions and feedback from execution, are required for the model to be able to adapt the intent. This, combined with the hierarchical approach, allows the system to generalize and adapt to any topology on its own and does not require training for a given network. Hence, providing LLMs with detailed, and well-structured information is critical. Without this, their ability to accurately interpret and act is significantly diminished, underscoring the importance of **well-curated, standardized datasets**. In fact, standardized KPIs for assurance are critical

to ensure that LLMs, whether through context learning, RAG, or fine-tuning, can effectively understand and manage KPIs. *With universally agreed-upon standard KPIs, LLMs don't have to "discover" or interpret the KPIs and metrics, thus reducing complexity and increasing the model's ability to accurately process, compare, and act on KPI data.*

C. Accuracy, Robustness, and Trustworthiness

To ensure the integrity and data quality of the generated policies we perform several checks as follows. A sanity check as part of the validation helps detect missing and incorrect values and ensure that the action sequence of the policy tree is correct and that the tree accurately represents the intent. Verification helps to ensure that the generated policies result in the desired state. For this, we need to evaluate the policies in a digital twin setting, where we can measure any difference between the deployed intent and desired state. Having a digital twin also helps with optimizations and testing different deployments. Last, to ensure consistency, our policy framework supports conflict detection and resolution across the abstraction hierarchy. For assurance policies, LLMs need to be accurate and define actions that bring the operational state closer to the target state. *Considering complex intents and scenarios where mathematical decision-making is crucial for maintaining performance, a hybrid approach that combines LLMs with specialized control algorithms or solvers such as optimizers can significantly enhance the effectiveness and reliability of the assurance actions.* In this regard, the LLM analyzes the relevant data, such as policies, KPIs and metrics, and when drift occurs, depending on the assurance action required, the LLM can call specialized algorithms to perform complex computations and then translate the computed solution into actionable policies. In contrast, for straightforward tasks, such as restart, redeploy, or adjust configuration, the LLM can output the policies independently.

D. Root Cause Analysis

By establishing a structured process for monitoring and assessing KPIs, we can detect intent drift, and analyze the root cause behind the deviation. This can become a starting point for deeper investigations, for example, whether there are configuration issues, resource allocation, or external factors. For instance, if CPU utilization consistently exceeds target values, RCA can investigate causes like workload distributions, software inefficiencies, hardware limitations.

VII. CONCLUSIONS AND FUTURE WORK

To truly close the loop, and feed the negative signal, we assure intents by: formalizing and assessing KPIs, quantifying intent drift, and then, for complex computations, using proven methods, such as controllers, to derive the actions. To enhance AI applications in network management, we discussed the limitations and opportunities towards trustworthy AI for network management. In future work, we plan to enhance the controller design to handle more complex scenarios and a wider variety of intents, as well as assurance at scale through global level optimizers and controllers that supervise multiple intents.

REFERENCES

- [1] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, "Intent-Based Networking - Concepts and Definitions," RFC 9315, Oct. 2022. [Online]. Available: <https://www.rfc-editor.org/info/rfc9315>
- [2] K. Dzevaroska, N. Beigi-Mohammadi, A. Tizghadam, and A. Leon-Garcia, "Towards a self-driving management system for the automated realization of intents," *IEEE Access*, vol. 9, pp. 159 882–159 907, 2021.
- [3] K. Dzevaroska, J. Lin, A. Tizghadam, and A. Leon-Garcia, "Llm-based policy generation for intent-based management of applications," in *2023 19th International Conference on Network and Service Management (CNSM)*. IEEE, 2023, pp. 1–7.
- [4] K. Dzevaroska, A. Tizghadam, and A. Leon-Garcia, "Emergence: An intent fulfillment system," *IEEE Communications Magazine*, 2024.
- [5] K. Dzevaroska, A. Tizghadam, and A. Leon-Garcia, "Intent assurance using llms guided by intent drift," in *NOMS 2024-2024 IEEE Network Operations and Management Symposium*. IEEE, 2024, pp. 1–7.
- [6] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "Netconfeval: Can llms facilitate network configuration?" *Proceedings of the ACM on Networking*, vol. 2, no. CoNEXT2, pp. 1–25, 2024.
- [7] B. Ifland, E. Duani, R. Krief, M. Ohana, A. Zilberman, A. Murillo, O. Manor, O. Lavi, H. Kenji, A. Shabtai *et al.*, "Genet: A multi-modal llm-based co-pilot for network topology and configuration," *arXiv preprint arXiv:2407.08249*, 2024.
- [8] C. Wang, M. Scazzariello, A. Farshin, D. Kostic, and M. Chiesa, "Making network configuration human friendly," *arXiv preprint arXiv:2309.06342*, 2023.
- [9] A. Fuad, A. H. Ahmed, M. A. Riegler, and T. Čičić, "An intent-based networks framework based on large language models," in *2024 IEEE 10th International Conference on Network Softwarization (NetSoft)*. IEEE, 2024, pp. 7–12.
- [10] E.-D. Jeong, H.-G. Kim, S. Nam, J.-H. Yoo, and J. W.-K. Hong, "Switch: Switch configuration assistant with llm and prompt engineering," in *NOMS 2024-2024 IEEE Network Operations and Management Symposium*. IEEE, 2024, pp. 1–7.
- [11] A. Mekrache, A. Ksentini, and C. Verikoukis, "Intent-based management of next-generation networks: an llm-centric approach," *IEEE Network*, 2024.
- [12] M. A. Habib, P. E. I. Rivera, Y. Ozcan, M. Elsayed, M. Bavand, R. Gaigalas, and M. Erol-Kantarci, "Llm-based intent processing and network optimization using attention-based hierarchical reinforcement learning," *arXiv preprint arXiv:2406.06059*, 2024.
- [13] N. Van Tu, J.-H. Yoo, and J. W.-K. Hong, "Towards intent-based configuration for network function virtualization using in-context learning in large language models," in *NOMS 2024-2024 IEEE Network Operations and Management Symposium*. IEEE, 2024, pp. 1–8.
- [14] D. Wang, Y. Wang, X. Jiang, Y. Zhang, Y. Pang, and M. Zhang, "When large language models meet optical networks: Paving the way for automation," *Electronics*, vol. 13, no. 13, p. 2529, 2024.
- [15] B. Martini, M. Gharbaoui, and P. Castoldi, "Intent-based network slicing for sdn vertical services with assurance: Context, design and preliminary experiments," *Future Generation Computer Systems*, vol. 142, pp. 101–116, 2023.
- [16] B. Claise, J. Quilbeuf, D. Lopez, D. Voyer, and T. Arumugam, "Service Assurance for Intent-Based Networking Architecture," RFC 9417, Jul. 2023. [Online]. Available: <https://www.rfc-editor.org/info/rfc9417>
- [17] B. Claise, J. Quilbeuf, P. Lucente, D. P. Fasano, and T. Arumugam, "A YANG Data Model for Service Assurance," RFC 9418, Jul. 2023. [Online]. Available: <https://www.rfc-editor.org/info/rfc9418>
- [18] A. Leivadeas and M. Falkner, "A survey on intent based networking," *IEEE Communications Surveys & Tutorials*, 2022.
- [19] A. Leivadeas and F. Matthias, "Autonomous network assurance in intent based networking: Vision and challenges," pp. 1–10, 2023.
- [20] R. Mohamed, I. Lambadaris, A. Leivadeas, J. Chinneck, T. Morris, and P. Djukic, "Automatic feasibility restoration for 5g cloud gaming," in *ICC 2023-IEEE International Conference on Communications*. IEEE, 2023, pp. 846–851.
- [21] V. S. Mai, R. J. La, T. Zhang, and A. Battou, "End-to-end quality-of-service assurance with autonomous systems: 5g/6g case study," in *2022 IEEE 19th Annual Consumer Communications & Networking Conference (CCNC)*. IEEE, 2022, pp. 644–651.
- [22] K. Edeline, T. Carlisi, J. Iurman, B. Claise, and B. Donnet, "Towards a closed-looped automation for service assurance with the dxagent," in *2022 IEEE 8th International Conference on Network Softwarization (NetSoft)*, 2022, pp. 67–72.
- [23] M. Falkner and J. Apostolopoulos, "Intent-based networking for the enterprise: a modern network architecture," *Communications of the ACM*, vol. 65, no. 11, pp. 108–117, 2022.
- [24] K. Samdanis, A. N. Abbou, J. Song, and T. Taleb, "Ai/ml service enablers & model maintenance for beyond 5g networks," *IEEE Network*, 2023.
- [25] S. K. Perepu, J. P. Martins, R. Souza, and K. Dey, "Intent-based multi-agent reinforcement learning for service assurance in cellular networks," in *GLOBECOM 2022-2022 IEEE Global Communications Conference*. IEEE, 2022, pp. 2879–2884.
- [26] J. Wang, J. Liu, J. Li, and N. Kato, "Artificial intelligence-assisted network slicing: Network assurance and service provisioning in 6g," *IEEE Vehicular Technology Magazine*, vol. 18, no. 1, pp. 49–58, 2023.
- [27] X. Zheng and A. Leivadeas, "Network assurance in intent-based networking data centers with machine learning techniques," in *2021 17th International Conference on Network and Service Management (CNSM)*. IEEE, 2021, pp. 14–20.
- [28] C. E. Rothenberg, D. A. L. Perez, N. F. S. de Sousa, R. V. Rosa, R. U. Mustafa, M. T. Islam, and P. H. Gomes, "Intent-based control loop for dash video service assurance using ml-based edge qoe estimation," in *2020 6th IEEE conference on network softwarization (NetSoft)*. IEEE, 2020, pp. 353–355.
- [29] L. Velasco, S. Barzegar, D. Sequeira, A. Ferrari, N. Costa, V. Curri, J. Pedro, A. Napoli, and M. Ruiz, "Autonomous and energy efficient lighthpath operation based on digital subcarrier multiplexing," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 9, pp. 2864–2877, 2021.
- [30] S. Barzegar, M. Ruiz, and L. Velasco, "Reinforcement learning-based autonomous multilayer network operation," in *2020 European Conference on Optical Communications (ECOC)*. IEEE, 2020, pp. 1–4.
- [31] F. Tabatabaeimehr, M. Ruiz, and L. Velasco, "Supporting beyond 5g applications by coordinating ai-based intent operation. an example for multilayer metro networks," in *2020 European Conference on Optical Communications (ECOC)*. IEEE, 2020, pp. 1–4.
- [32] T. A. Khan, A. Mehmood, J. J. D. Rivera, and W.-C. Song, "Machine learning approach for automatic configuration and management of 5g platforms," in *2019 20th Asia-Pacific Network Operations and Management Symposium (APNOMS)*. IEEE, 2019, pp. 1–6.
- [33] K. Abbas, M. Afaq, T. A. Khan, A. Mehmood, and W.-C. Song, "Ibnslicing: Intent-based network slicing framework for 5g networks using deep learning," in *2020 21st Asia-Pacific Network Operations and Management Symposium (APNOMS)*. IEEE, 2020, pp. 19–24.
- [34] H. Wang, A. Abhashkumar, C. Lin, T. Zhang, X. Gu, N. Ma, C. Wu, S. Liu, W. Zhou, Y. Dong *et al.*, "{NetAssistant}: Dialogue based network diagnosis in data center networks," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 2024, pp. 2011–2024.
- [35] K. Dzevaroska, "Intent-based network management," Ph.D. dissertation, University of Toronto, 2024.
- [36] L. Baresi, S. Guinea, A. Leva, and G. Quattrocchi, "A discrete-time feedback controller for containerized cloud applications," in *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2016, pp. 217–228.
- [37] B. Chesneau and contributors, "Gunicorn: Python wsgi http server for unix," 2023, accessed: 2024-09-06. [Online]. Available: <https://gunicorn.org/>
- [38] Apache Software Foundation, "Apache jmeter," 2023, accessed: 2024-09-06. [Online]. Available: <https://jmeter.apache.org/>
- [39] K. Dzevaroska, P. Varga, K. Tutschku, and S. Jang, "Generative ai for network management," Panel discussion at the IEEE/IFIP Network Operations and Management Symposium (NOMS 2024), May 2024, panelist. [Online]. Available: <https://noms2024.ieee-noms.org/program/panels-sessions>